Buku Panduan
& Source Code

Jenis Ciptaan

Perangkat Lunak

Judul Ciptaan

CipherVision
(Analisis Frekuensi dan Visualisasi Cipher Substitusi)

Pencipta :

Daniel Aquaries / A11.2023.15003

Fauzi Adi Rafrastara, M.CS. / 0686.11.2011.398

Arvin Farrel Pramuditya / A11.2023.15062

Ramadhan Rakhmat Sani /

Universitas Dian Nuswantoro

Semarang

2025

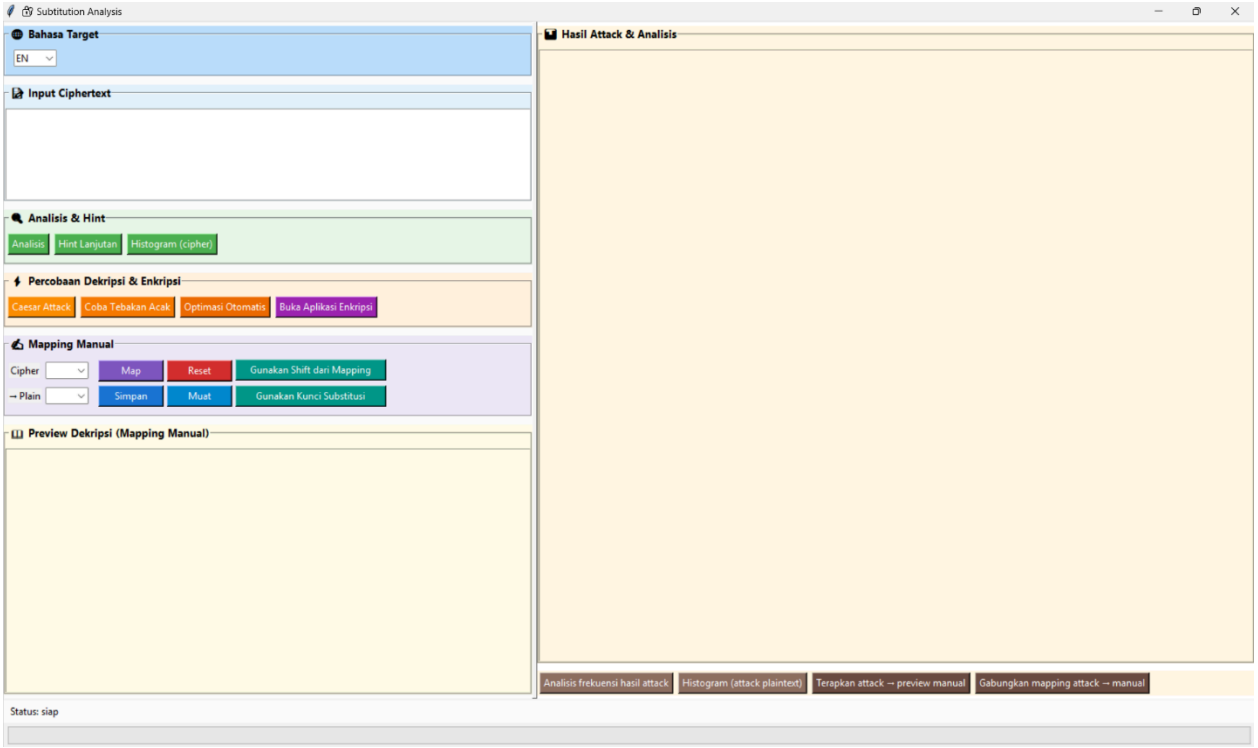
Daftar Isi

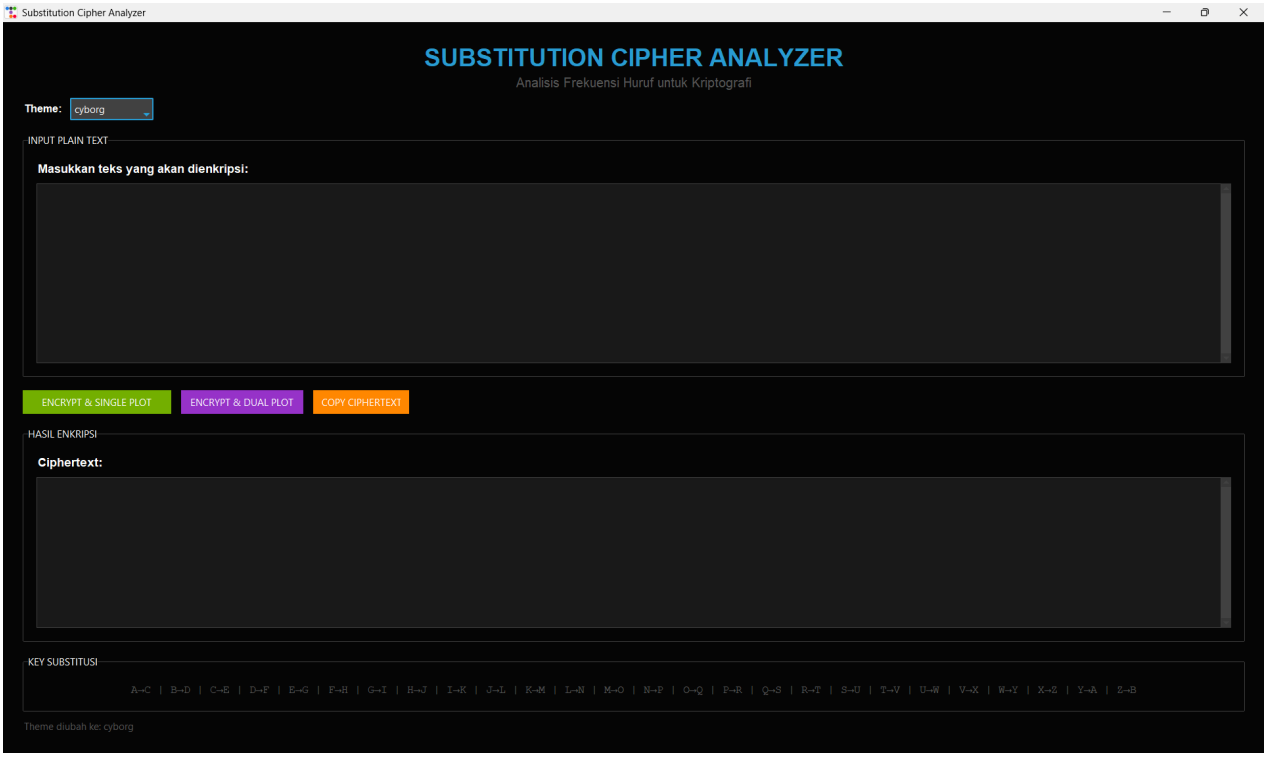
Daftar Isi..... 2

1. Gambaran Aplikasi..... 3
2. Manual Penggunaan..... 4
   A. Aplikasi analisis CipherVision..... 4
   B. Aplikasi Enkripsi..... 6
      1). Fitur Utama..... 8
      2). Kebutuhan SIstem..... 8
3. Source Code Analisis Cipher (analysis_frequent_text.py)..... 10
4. Source Code Analisis Enkripsi (encrypth_text.py)..... 10

1. Gambaran Aplikasi

Tampilan awal aplikasi CipherVision





Keterangan :

CipherVision terdiri dari 2 aplikasi terintegrasi:

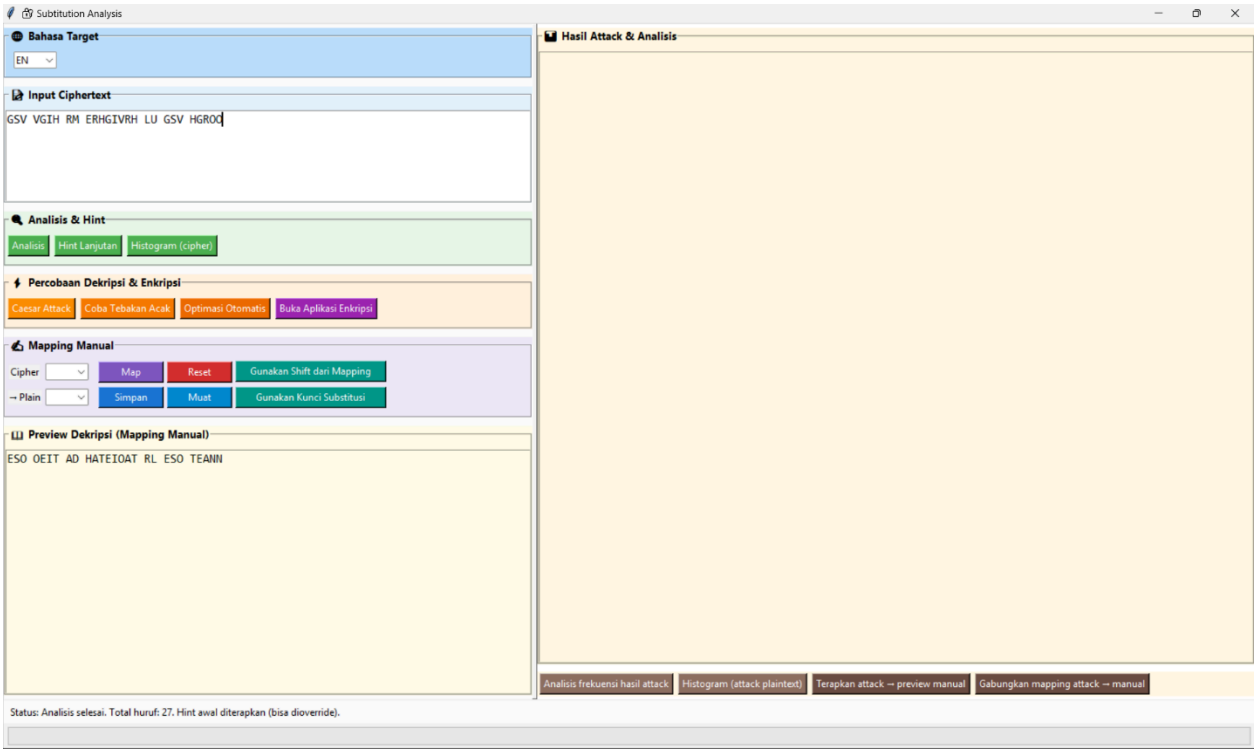
1. Aplikasi Analisis Cipher (analysis_frequent_text.py)
    - Alat dekripsi cipher substitusi
    - Analisis frekuensi huruf

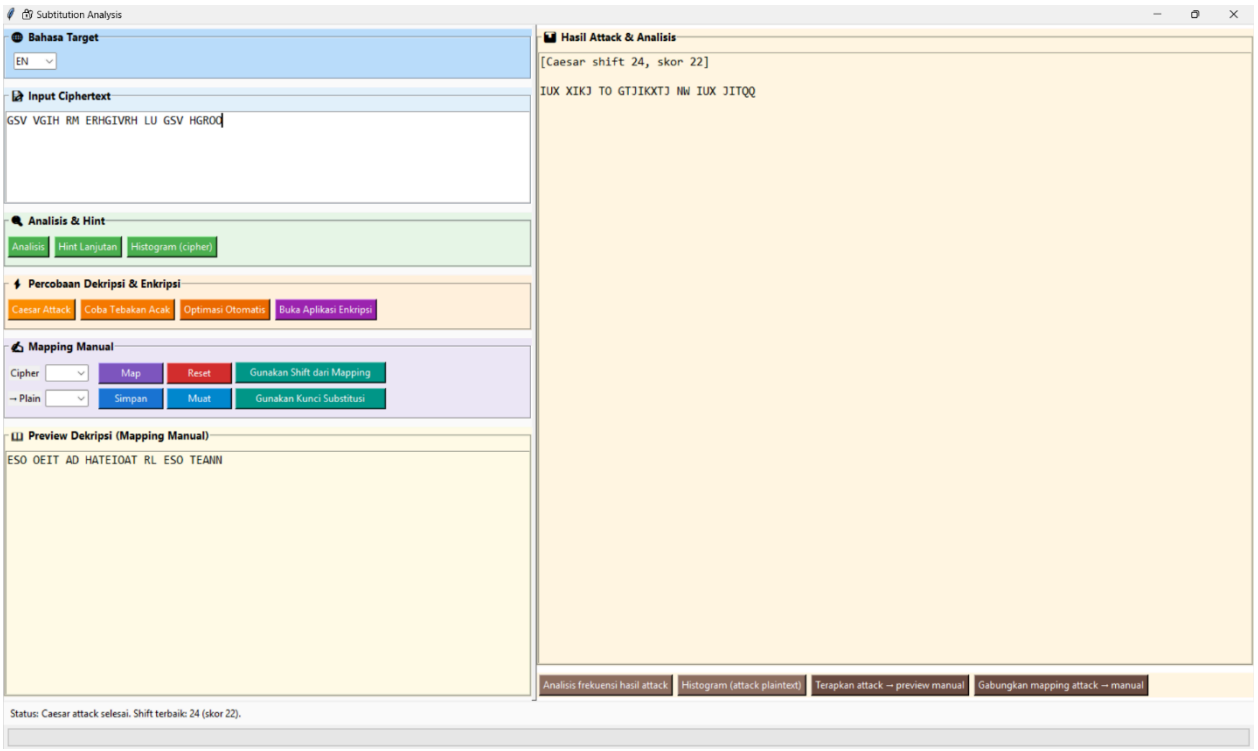
    - Multiple attack methods (Caesar, Random, Auto-Tune)
    - Dukungan bahasa Inggris dan Indonesia
2. Aplikasi Enkripsi & Visualisasi (encrypth_text.py)
    - Generator cipher substitusi
    - Visualisasi frekuensi (single/dual plot)
    - Interface modern dengan tema customizable

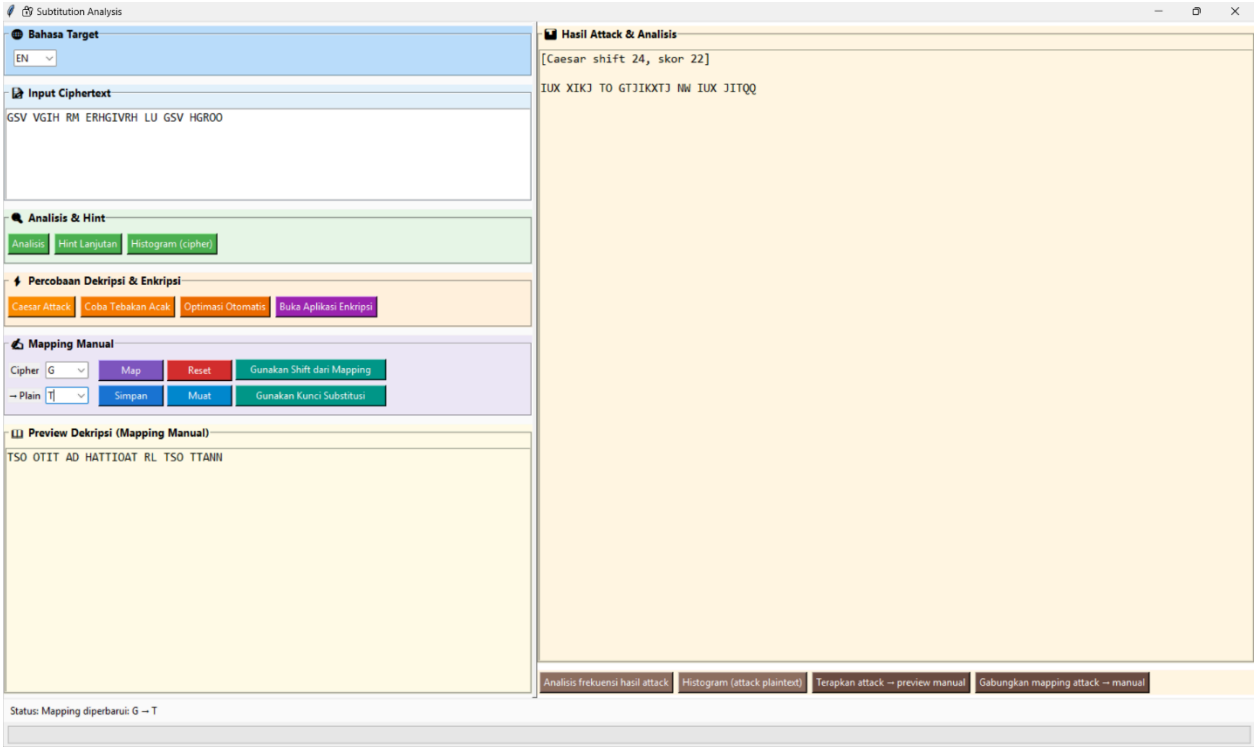
2. Manual Penggunaan

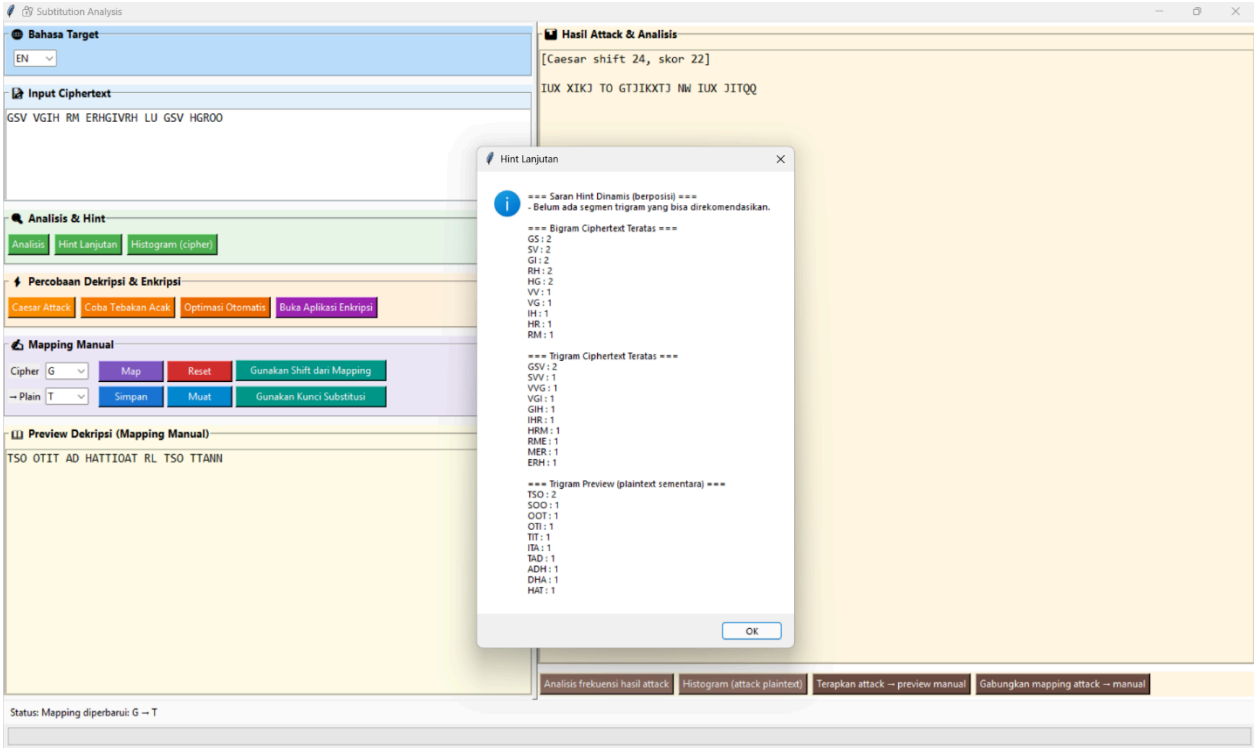
A. Aplikasi analisis CipherVision

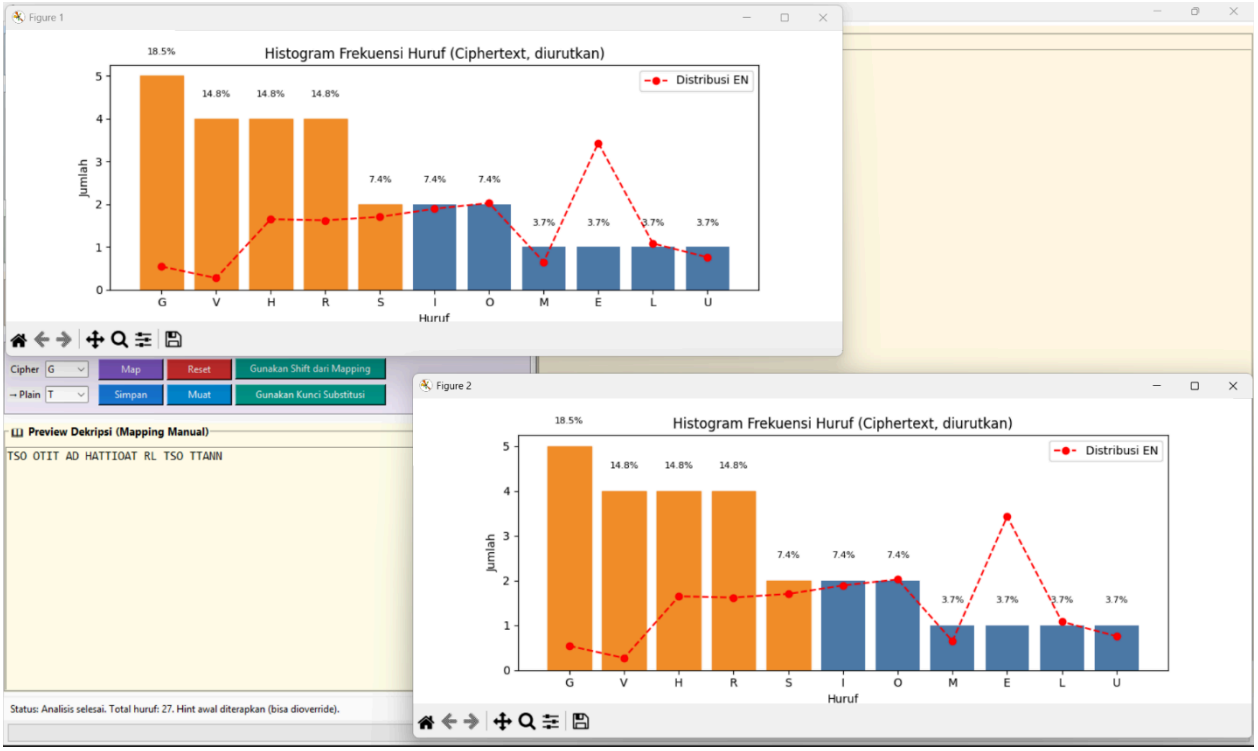
1. **Jalankan:** python analysis_frequent_text.py
2. **Langkah:**
    - Masukkan ciphertext di panel kiri
    - Pilih bahasa target (EN/ID)
    - Klik "Analisis" untuk mapping awal
    - Gunakan metode attack:
        - Caesar Attack (otomatis/manual)
        - Random Attack (8000-20000 iterasi)
        - Auto-Tune Attack
    - Atur mapping manual jika diperlukan
    - Hasil muncul di panel kanan


Subtitution Analysis
Bahasa Target
EN
Input Ciphertext
GSV VGIH RM ERHGIVRH LU GSV HGROO
Analisis & Hint
Analisis
Hint Lanjutan
Histogram (cipher)
Percobaan Dekripsi & Enkripsi
Caesar Attack
Coba Tebakan Acak
Optimasi Otomatis
Buka Aplikasi Enkripsi
Mapping Manual
Cipher
→ Plain
Map
Reset
Gunakan Shift dari Mapping
Simpan
Muat
Gunakan Kunci Substitusi
Preview Dekripsi (Mapping Manual)
ESO OEIT AD HATEIOAT RL ESO TEANN
Hasil Attack & Analisis
Analisis frekuensi hasil attack
Histogram (attack plaintext)
Terapkan attack → preview manual
Gabungkan mapping attack → manual
Status: Analisis selesai. Total huruf: 27. Hint awal diterapkan (bisa dioverride).


Subtitution Analysis
Bahasa Target
EN
Input Ciphertext
GSV VGIH RM ERHGIVRH LU GSV HGROO
Analisis & Hint
Analisis
Hint Lanjutan
Histogram (cipher)
Percobaan Dekripsi & Enkripsi
Caesar Attack
Coba Tebakan Acak
Optimasi Otomatis
Buka Aplikasi Enkripsi
Mapping Manual
Cipher
→ Plain
Map
Reset
Gunakan Shift dari Mapping
Simpan
Muat
Gunakan Kunci Substitusi
Preview Dekripsi (Mapping Manual)
ESO OEIT AD HATEIOAT RL ESO TEANN
Hasil Attack & Analisis
[Caesar shift 24, skor 22]
IUX XIKJ TO GTJIKXTJ NW IUX JITQQ
Analisis frekuensi hasil attack
Histogram (attack plaintext)
Terapkan attack → preview manual
Gabungkan mapping attack → manual
Status: Caesar attack selesai. Shift terbaik: 24 (skor 22).

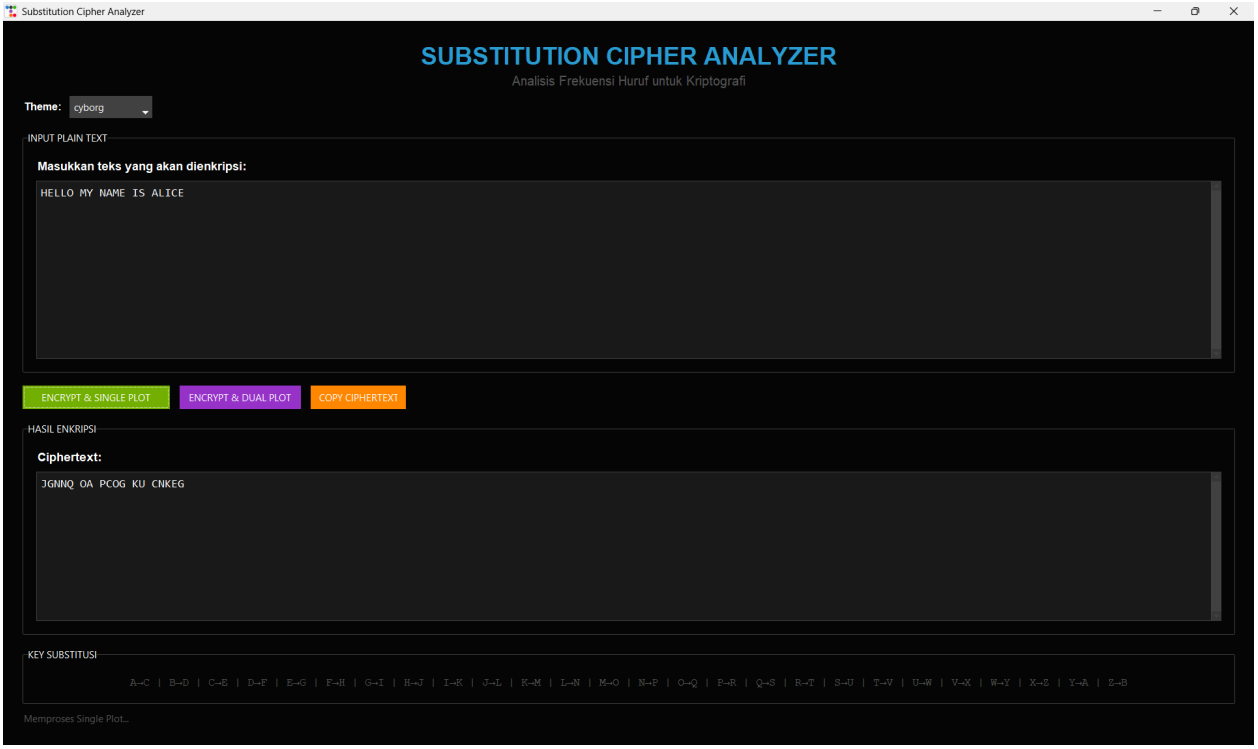

Subtitution Analysis
Bahasa Target
EN
Input Ciphertext
GSV VGIH RM ERHGIVRH LU GSV HGROO
Analisis & Hint
Analisis
Hint Lanjutan
Histogram (cipher)
Percobaan Dekripsi & Enkripsi
Caesar Attack
Coba Tebakan Acak
Optimasi Otomatis
Buka Aplikasi Enkripsi
Mapping Manual
Cipher G
Map
Reset
Gunakan Shift dari Mapping
→ Plain T
Simpan
Muat
Gunakan Kunci Substitusi
Preview Dekripsi (Mapping Manual)
TSO OTIT AD HATTIOAT RL TSO TTANN
Hasil Attack & Analisis
[Caesar shift 24, skor 22]
IUX XIKJ TO GTJIKXTJ NW IUX JITQQ
Analisis frekuensi hasil attack
Histogram (attack plaintext)
Terapkan attack → preview manual
Gabungkan mapping attack → manual
Status: Mapping diperbarui: G → T

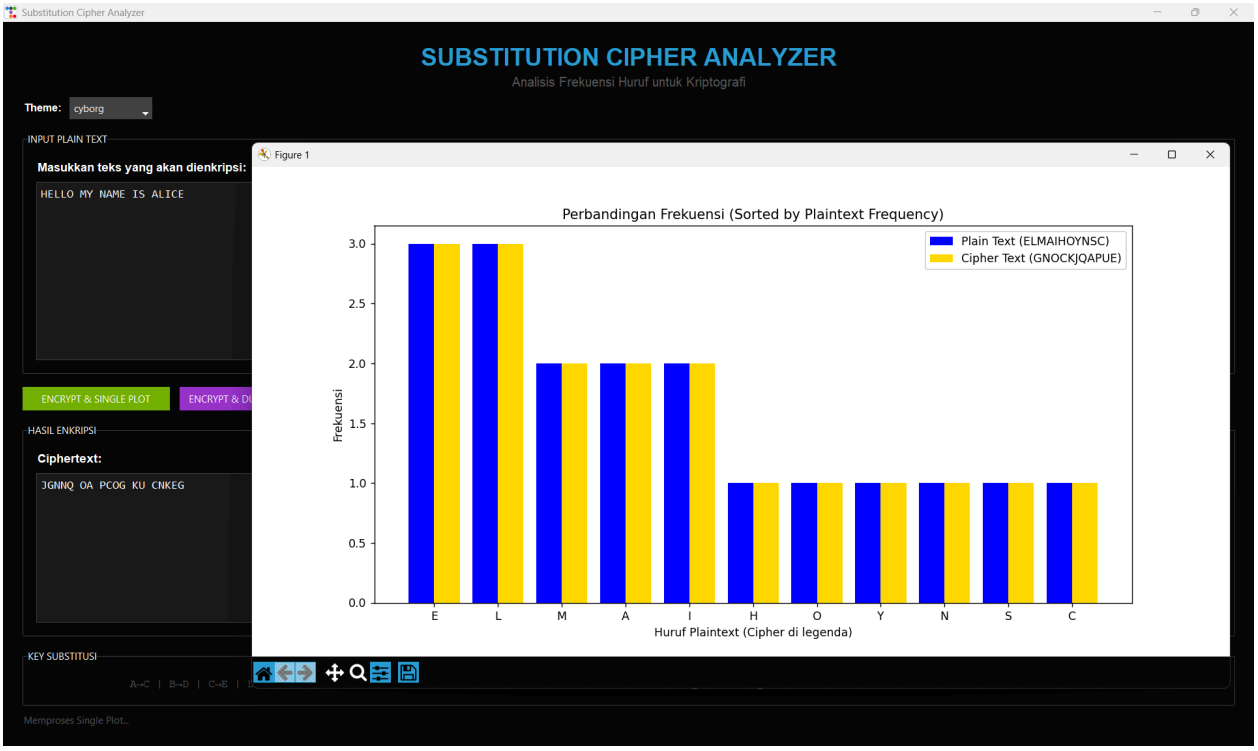

Subtitution Analysis
Bahasa Target
EN
Input Ciphertext
GSV VGIH RM ERHGIVRH LU GSV HGROO
Analisis & Hint
Analisis
Hint Lanjutan
Histogram (cipher)
Percobaan Dekripsi & Enkripsi
Caesar Attack
Coba Tebakan Acak
Optimasi Otomatis
Buka Aplikasi Enkripsi
Mapping Manual
Cipher G
Map
Reset
Gunakan Shift dari Mapping
→ Plain T
Simpan
Muat
Gunakan Kunci Substitusi
Preview Dekripsi (Mapping Manual)
TSO OTIT AD HATTIOAT RL TSO TTANN
Hasil Attack & Analisis
[Caesar shift 24, skor 22]
IUX XIKJ TO GTJIKXTJ NW IUX JITQQ
Hint Lanjutan
=== Saran Hint Dinamis (berposisi) ===
- Belum ada segmen trigram yang bisa direkomendasikan.
=== Bigram Ciphertext Teratas ===
GS : 2
SV : 2
GI : 2
RH : 2
HG : 2
VV : 1
VG : 1
IH : 1
HR : 1
RM : 1
=== Trigram Ciphertext Teratas ===
GSV : 2
SVV : 1
VVG : 1
VGI : 1
GIH : 1
IHR : 1
HRM : 1
RME : 1
MER : 1
ERH : 1
=== Trigram Preview (plaintext sementara) ===
TSO : 2
SOO : 1
OOT : 1
OTI : 1
TIT : 1
ITA : 1
TAD : 1
ADH : 1
DHA : 1
HAT : 1
OK
Analisis frekuensi hasil attack
Histogram (attack plaintext)
Terapkan attack → preview manual
Gabungkan mapping attack → manual
Status: Mapping diperbarui: G → T

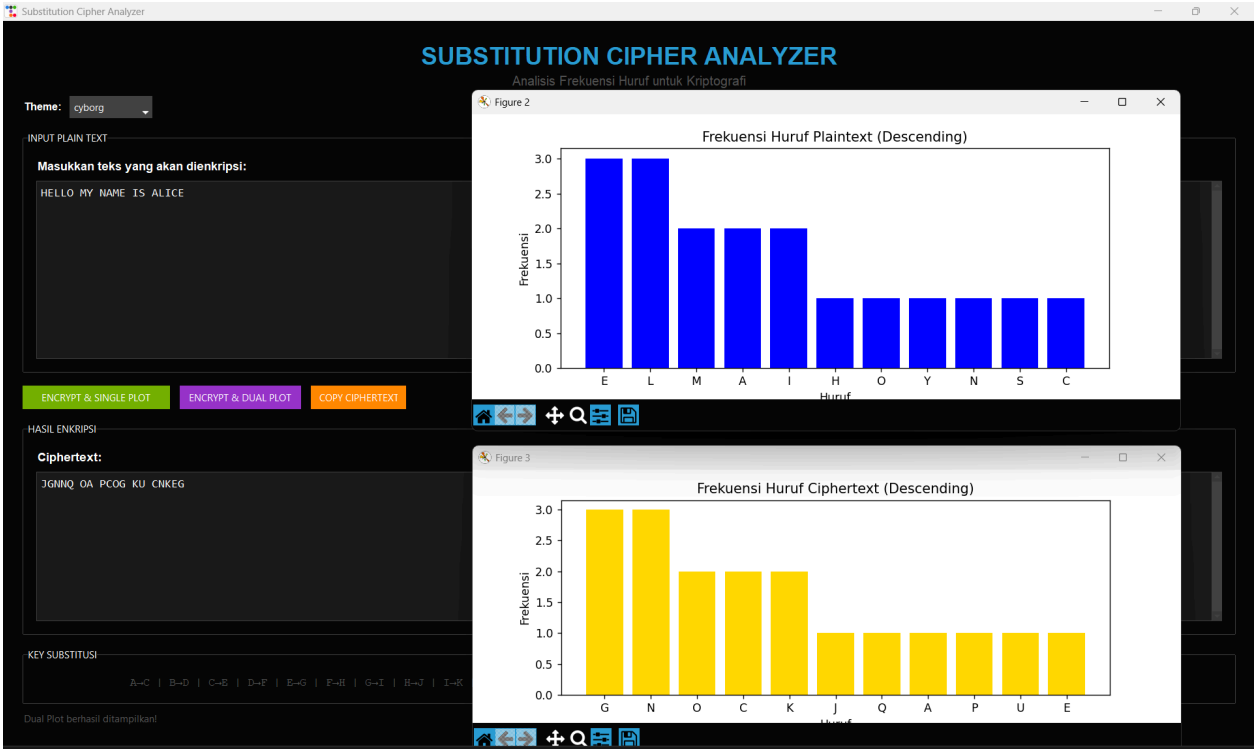


B. Aplikasi Enkripsi

1. **Jalankan:** python encrypth_text.py
2. **Langkah:**
    - Masukkan plaintext
    - Klik "ENCRYPT & SINGLE PLOT" (grafik gabungan)
    - Atau "ENCRYPT & DUAL PLOT" (2 grafik terpisah)
    - Ciphertext muncul di panel hasil
    - Dapat di-copy ke clipboard


Substitution Cipher Analyzer
SUBSTITUTION CIPHER ANALYZER
Analisis Frekuensi Huruf untuk Kriptografi
Theme:
cyborg
INPUT PLAIN TEXT
Masukkan teks yang akan dienkripsi:
HELLO MY NAME IS ALICE
ENCRYPT & SINGLE PLOT
ENCRYPT & DUAL PLOT
COPY CIPHERTEXT
HASIL ENKRIPSI
Ciphertext:
JGNNQ OA PCOG KU CNKEG
KEY SUBSTITUSI
A→C | B→D | C→E | D→F | E→G | F→H | G→I | H→J | I→K | J→L | K→M | L→N | M→O | N→P | O→Q | P→R | Q→S | R→T | S→U | T→V | U→W | V→X | W→Y | X→Z | Y→A | Z→B
Memproses Single Plot...


Substitution Cipher Analyzer
SUBSTITUTION CIPHER ANALYZER
Analisis Frekuensi Huruf untuk Kriptografi
Theme:
cyborg
INPUT PLAIN TEXT
Masukkan teks yang akan dienkripsi:
HELLO MY NAME IS ALICE
ENCRYPT & SINGLE PLOT
HASIL ENKRIPSI
Ciphertext:
JGNNQ OA PCOG KU CNKEG
KEY SUBSTITUSI
Memproses Single Plot...
Figure 1
Perbandingan Frekuensi (Sorted by Plaintext Frequency)
Plain Text (ELMAIHOYNSC)
Cipher Text (GNOCKJQAPUE)
Frekuensi
3.0
2.5
2.0
1.5
1.0
0.5
0.0
E
L
M
A
I
H
O
Y
N
S
C
Huruf Plaintext (Cipher di legenda)



1). Fitur Utama

- *Analisis frekuensi multi-bahasa*
- *Hint dinamis berdasarkan posisi*
- *Visualisasi distribusi huruf*
- *Save/load mapping ke JSON*
- *Percobaan Dekripsi dengan Caesar dan Tebakan Acak dengan Perulangan*
- *Integration antar aplikasi*

2). Kebutuhan Sistem

- Halaman Python 3.7+
- Library: tkinter, matplotlib, numpy, ttkbootstrap
- Sistem operasi: Windows/Linux/macOS

Source Code

Aplikasi Cipher Vision (Analisis Frekuensi dan Visualisasi Cipher Subtitusi)

3. Source Code Analisis Cipher (analysis_frequent_text.py)

a. Running App

```
if __name__ == "__main__":
    root = tk.Tk()
    app = SubstitutionGUI(root)
    root.mainloop()
```

b. Method-Method untuk analisa

```
def normalize_text(text):
    return "".join(ch for ch in text.upper() if ch in string.ascii_uppercase)


def frequency_analysis(text):
    norm = normalize_text(text)
    total = len(norm)
    counter = collections.Counter(norm)
    return counter, total


def proportions(counter, total):
    return {
        k: (counter.get(k, 0) / total * 100 if total > 0 else 0.0)
        for k in string.ascii_uppercase
    }


def suggest_mapping(counter):
    cipher_order = [l for l, _ in sorted(counter.items(), key=lambda x: (-x[1],
x[0]))]
    suggestion = {}
    for i, c in enumerate(cipher_order):
        if i < len(ENGLISH_FREQ_ORDER):
            suggestion[c] = ENGLISH_FREQ_ORDER[i]
    return suggestion


def apply_mapping(text, mapping, unknown_char="·"):
    out = []
    for ch in text:
        up = ch.upper()
        if up in string.ascii_uppercase:
            mapped = mapping.get(up)
            if mapped:
                out.append(mapped if ch.isupper() else mapped.lower())
            else:
```

```
                out.append(unknown_char)
        else:
            out.append(ch)
    return "".join(out)
```

c. Analisis Cipher Text

```
 def analyze(self):
        self.ciphertext = self.input_text.get("1.0", "end").strip()
        if not self.ciphertext:
            messagebox.showwarning("Peringatan", "Masukkan ciphertext terlebih
dahulu.")
            return
        counter, total = frequency_analysis(self.ciphertext)
        self.hint_mapping = suggest_mapping(counter)
        self.update_preview_manual()
        self.set_status(
            f"Analisis selesai. Total huruf: {total}. Hint awal diterapkan (bisa
dioverride)."
        )
```

d. Caesar Decrypt

```
def caesar_decrypt(ciphertext, shift):
    result = []
    for ch in ciphertext:
        if ch.isalpha():
            base = ord("A") if ch.isupper() else ord("a")
            result.append(chr((ord(ch) - base - shift) % 26 + base))
        else:
            result.append(ch)
    return "".join(result)
```

```
def run_caesar_attack(self):
        self.ciphertext = self.input_text.get("1.0", "end").strip()
        if not self.ciphertext:
            messagebox.showwarning("Peringatan", "Masukkan ciphertext terlebih
dahulu.")
            return

        # Tanya user apakah mau input shift manual
        choice = messagebox.askyesno(
            "Caesar Attack",
            "Apakah Anda ingin memasukkan shift secara manual?\n"
            "Ya = input manual, Tidak = cari otomatis terbaik.",
        )

        if choice:  # === Mode manual ===
```

```
            shift = self.ask_shift_with_slider()
            if shift is None:
                return

            preview = caesar_decrypt(self.ciphertext, shift)
            self.attack_preview = preview
            self.attack_mapping = {}
            self.attack_text.delete("1.0", "end")
            self.attack_text.insert(
                "1.0", f"[Caesar shift {shift} (manual via
slider)]\n\n{self.attack_preview}"
            )
            self.set_status(f"Preview Caesar dengan shift {shift} ditampilkan.")

        else:  # === Mode otomatis (brute force terbaik) ===
            best_shift, best_text, best_score, _ = caesar_attack(
                self.ciphertext, lang=self.lang.get()
            )
            self.attack_preview = best_text
            self.attack_mapping = {}
            self.attack_text.delete("1.0", "end")
            self.attack_text.insert(
                "1.0",
                f"[Caesar shift {best_shift}, skor
{best_score}]\n\n{self.attack_preview}",
            )
            self.set_status(
                f"Caesar attack selesai. Shift terbaik: {best_shift} (skor
{best_score})."
            )
```

e. Mapping Cipher Text to Plain Text

```
def update_mapping(self):
        c = self.cipher_var.get().upper()
        p = self.plain_var.get().upper()
        if c in string.ascii_uppercase and p in string.ascii_uppercase:
            self.mapping[c] = p
            self.update_preview_manual()
            self.set_status(f"Mapping diperbarui: {c} → {p}")
        else:
            messagebox.showerror("Error", "Pilih huruf A–Z untuk mapping.")

    def reset_mapping(self):
        self.mapping = {}
        self.update_preview_manual()
        self.set_status("Mapping manual dikosongkan (hint awal tetap ada).")
```

```
    def save_mapping(self):
        if not self.mapping:
            messagebox.showinfo("Info", "Mapping manual kosong.")
            return
        path = filedialog.asksaveasfilename(
            defaultextension=".json", filetypes=[("JSON", "*.json")]
        )
        if not path:
            return
        with open(path, "w", encoding="utf-8") as f:
            json.dump(self.mapping, f, ensure_ascii=False, indent=2)
        self.set_status(f"Mapping disimpan ke {path}")

    def load_mapping(self):
        path = filedialog.askopenfilename(filetypes=[("JSON", "*.json")])
        if not path:
            return
        try:
            with open(path, "r", encoding="utf-8") as f:
                data = json.load(f)
            for k, v in data.items():
                if k not in string.ascii_uppercase or v not in
string.ascii_uppercase:
                    raise ValueError("Format mapping tidak valid.")
            self.mapping = data
            self.update_preview_manual()
            self.set_status(f"Mapping dimuat dari {path}")
        except Exception as e:
            messagebox.showerror("Error", f"Gagal memuat mapping: {e}")
```

f. Show Histogram Frequency

```
def show_histogram_cipher(self):
        if not self.ciphertext:
            messagebox.showwarning(
                "Peringatan", "Analisis dulu sebelum menampilkan histogram."
            )
            return

        counter, total = frequency_analysis(self.ciphertext)

        # Sort berdasarkan frekuensi
        items = sorted(counter.items(), key=lambda x: -x[1])
        letters = [l for l, _ in items]
        values = [counter[l] for l in letters]
```

```python
        plt.figure(figsize=(12, 5))
        bars = plt.bar(letters, values, color="#4C78A8")

        # Mewarnai top 5
        for i, bar in enumerate(bars):
            if i < 5:
                bar.set_color("#f28e2b")

        # Label persentase
        for i, v in enumerate(values):
            plt.text(i, v + 0.5, f"{v/total*100:.1f}%", ha="center", fontsize=8)

        plt.title("Histogram Frekuensi Huruf (Ciphertext, diurutkan)")
        plt.xlabel("Huruf")
        plt.ylabel("Jumlah")

        # Gambaran distribusi bahasa target
        lang = self.lang.get().upper()
        if lang == "EN":
            english_freq = {
                "E": 12.7,
                "T": 9.1,
                "A": 8.2,
                "O": 7.5,
                "I": 7.0,
                "N": 6.7,
                "S": 6.3,
                "H": 6.1,
                "R": 6.0,
                "D": 4.3,
                "L": 4.0,
                "C": 2.8,
                "U": 2.8,
                "M": 2.4,
                "W": 2.4,
                "F": 2.2,
                "G": 2.0,
                "Y": 2.0,
                "P": 1.9,
                "B": 1.5,
                "V": 1.0,
                "K": 0.8,
                "J": 0.15,
                "X": 0.15,
```

```
                "Q": 0.1,
                "Z": 0.07,
            }
            ref_values = [english_freq.get(l, 0) / 100 * total for l in letters]
            plt.plot(
                letters,
                ref_values,
                color="red",
                marker="o",
                linestyle="--",
                label="Distribusi EN",
            )
            plt.legend()

        elif lang == "ID":
            indonesian_freq = {
                "A": 19.0,
                "N": 15.0,
                "I": 9.0,
                "E": 9.0,
                "U": 8.0,
                "T": 7.0,
                "R": 6.0,
                "K": 6.0,
                "S": 5.0,
                "D": 4.0,
                "M": 4.0,
                "L": 3.0,
                "O": 3.0,
                "G": 2.0,
                "B": 2.0,
                "P": 2.0,
                "H": 1.0,
                "Y": 1.0,
                "J": 1.0,
                "C": 1.0,
                "W": 1.0,
                "F": 0.5,
                "Z": 0.5,
                "V": 0.2,
                "X": 0.1,
                "Q": 0.1,
            }
            ref_values = [indonesian_freq.get(l, 0) / 100 * total for l in
letters]
```

```
        plt.plot(
            letters,
            ref_values,
            color="red",
            marker="o",
            linestyle="--",
            label="Distribusi ID",
        )
        plt.legend()

    plt.show()
```

4. Source Code Analisis Enkripsi (encrypth_text.py)

g. Input Section

```
    # Input section
    input_frame = tb.Labelframe(
        main_container, text="INPUT PLAIN TEXT", padding=20)
    input_frame.pack(fill=tk.BOTH, expand=True, pady=(0, 20))

    tb.Label(input_frame, text="Masukkan teks yang akan dienkripsi:",
             font=("Arial", 11, "bold")).pack(anchor=tk.W, pady=(0, 10))

    # Text widget dengan scrollbar untuk input
    input_container = tb.Frame(input_frame)
    input_container.pack(fill=tk.BOTH, expand=True)

    self.entry = tb.Text(input_container, height=8,
                         font=("Consolas", 11), wrap=tk.WORD)
    scrollbar_input = tb.Scrollbar(
        input_container, orient=tk.VERTICAL, command=self.entry.yview)
    self.entry.configure(yscrollcommand=scrollbar_input.set)

    self.entry.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    scrollbar_input.pack(side=tk.RIGHT, fill=tk.Y)
```

h. Output Section

```
        # Result section
        result_frame = tb.Labelframe(
            main_container, text="HASIL ENKRIPSI", padding=20)
        result_frame.pack(fill=tk.BOTH, expand=True, pady=(0, 20))

        tb.Label(result_frame, text="Ciphertext:",
                 font=("Arial", 11, "bold")).pack(anchor=tk.W, pady=(0, 10))

        # Text widget dengan scrollbar untuk hasil
        result_container = tb.Frame(result_frame)
        result_container.pack(fill=tk.BOTH, expand=True)

        self.result_text = tb.Text(result_container, height=6, font=("Consolas",
11),
                                   foreground="#2ecc71", wrap=tk.WORD,
state="disabled")
        scrollbar_result = tb.Scrollbar(
            result_container, orient=tk.VERTICAL, command=self.result_text.yview)
        self.result_text.configure(yscrollcommand=scrollbar_result.set)

        self.result_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        scrollbar_result.pack(side=tk.RIGHT, fill=tk.Y)
```

i. Single Plot Section

```
        # --- Plot gabungan (single plot) - WARNA DIUBAH ---



def plot_single(plain_text, cipher_text):
    alphabet = list("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
    plain_counter = Counter([c for c in plain_text.upper() if c in alphabet])
    cipher_counter = Counter([c for c in cipher_text.upper() if c in alphabet])

    plain_sorted = sorted(plain_counter.items(),
                          key=lambda x: x[1], reverse=True)
    cipher_mapped = [
        (cipher_counter.get(generate_key()[ch], 0), generate_key()[ch])
        for ch, _ in plain_sorted
    ]

    plain_letters = [ch for ch, _ in plain_sorted]
    plain_freq = [freq for _, freq in plain_sorted]
    cipher_freq = [freq for freq, _ in cipher_mapped]
```

```python
    cipher_letters = [c for _, c in cipher_mapped]

    x = np.arange(len(plain_letters))
    width = 0.4

    plt.figure(figsize=(12, 6))
    plt.bar(
        x - width / 2,
        plain_freq,
        width,
        label="Plain Text (" + "".join(plain_letters) + ")",
        color="blue",  # DIUBAH: biru seperti dual plot
    )
    plt.bar(
        x + width / 2,
        cipher_freq,
        width,
        label="Cipher Text (" + "".join(cipher_letters) + ")",
        color="gold",  # DIUBAH: gold/kuning seperti dual plot
    )

    plt.xticks(x, plain_letters)
    plt.xlabel("Huruf Plaintext (Cipher di legenda)")
    plt.ylabel("Frekuensi")
    plt.title("Perbandingan Frekuensi (Sorted by Plaintext Frequency)")
    plt.legend()
    plt.show()
```